TRƯỜNG ĐẠI HỌC VĂN HIẾN

Thành Nhân trước thành Danh

MÔN HỆ ĐIỀU HÀNH

Giảng viên: ThS. Ngô Quốc Huy

Thời lượng: 30 tiết

CHƯƠNG 4: DEADLOCK

NỘI DUNG

- Giới thiệu về deadlock
- Các phương thức quản lý deadlock
- Phát hiện deadlock
- Ngăn chặn deadlock
- Tránh deadlock
- Phục hồi sau deadlock

Giới thiệu về deadlock

- Hệ thống bao gồm nhiều loại tài nguyên
- Ký hiệu tài nguyên: R1, R2, … , Rm
  - Chu kỳ CPU, không gian nhớ, thiết bị nhập/xuất…
- Mỗi tài nguyên R_i có W_i thực thể (instance)

Giới thiệu về deadlock

- Mỗi tiến trình sử dụng tài nguyên như sau:
  - Yêu cầu (request): tiến trình phải chờ nếu yêu cầu không được đáp ứng ngay
  - Sử dụng (use): tiến trình thao tác trên tài nguyên
  - Giải phóng (release): tiến trình hoàn trả lại tài nguyên

Giới thiệu về deadlock

- Tắc nghẽn (Deadlock): hai hoặc nhiều tiến trình đang chờ đợi vô hạn cho một sự kiện mà có thể được gây ra bởi chỉ một trong các tiến trình đang chờ

Giới thiệu về deadlock

- Ví dụ:
  - Cùng loại tài nguyên: Xét hệ thống với 3 ổ đĩa CD RW. Giả sử có 3 tiến trình và mỗi tiến trình giữ một ổ đĩa. Nếu mỗi tiến trình yêu cầu thêm một ổ đĩa khác thì 3 tiến trình sẽ rơi vào trạng thái tắc nghẽn. Mỗi tiến trình đang chờ sự kiện "ổ CD RW được giải phóng"
  - Khác loại tài nguyên: Xét hệ thống có 1 máy in và 1 ổ đĩa DVD. Giả sử tiến trình P_i đang giữ ổ DVD và tiến trình P_j đang giữ máy in. nếu P_i yêu cầu máy in và P_j yêu cầu ổ DVD thì sẽ xảy ra tắc nghẽn

Giới thiệu về deadlock

- Tắc nghẽn xuất hiện nếu 4 điều kiện sau xảy ra đồng thời

1. Độc quyền truy xuất (Mutual Exclusion): tại một thời điểm chỉ có duy nhất 1 tiến trình sử dụng tài nguyên
2. Giữ và đợi (Hold and Wait): một tiến trình giữ ít nhất 1 tài nguyên đang chờ thêm tài nguyên được giữ bởi tiến trình khác
3. Không trưng dụng (No preemption = No resource preemption): một tài nguyên có thể chỉ được giải phóng một cách tự nguyện bởi một tiến trình đang giữ nó, sau khi tiến trình đó hoàn thành tác vụ của mình

Giới thiệu về deadlock

- Tắc nghẽn xuất hiện nếu 4 điều kiện sau xảy ra đồng thời

4. Đợi vòng quanh (Circular wait): tồn tại một tập tiến trình đang chờ $\{P_0, P_1, \ldots, P_n\}$. Trong đó:

   P_0 đang chờ tài nguyên được giữ bởi P_1,

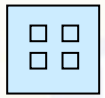
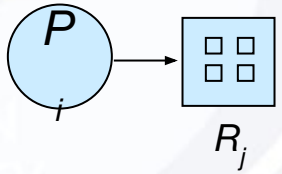
   P_1 đang chờ tài nguyên được giữ bởi P_2,

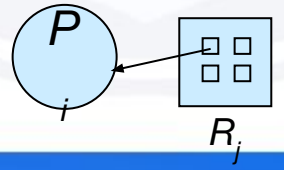
   … ,

   P_{n-1} đang chờ tài nguyên được giữ bởi P_n,

   P_n đang chờ tài nguyên được giữ bởi P_0.

Giới thiệu về deadlock

- Đồ thị cấp phát tài nguyên (RAG: Resource-Allocation Graph)
  - Tiến trình Pi
  - Tài nguyên với 4 thực thể Rj 
  - Pi yêu cầu thực thể tài nguyên Rj 

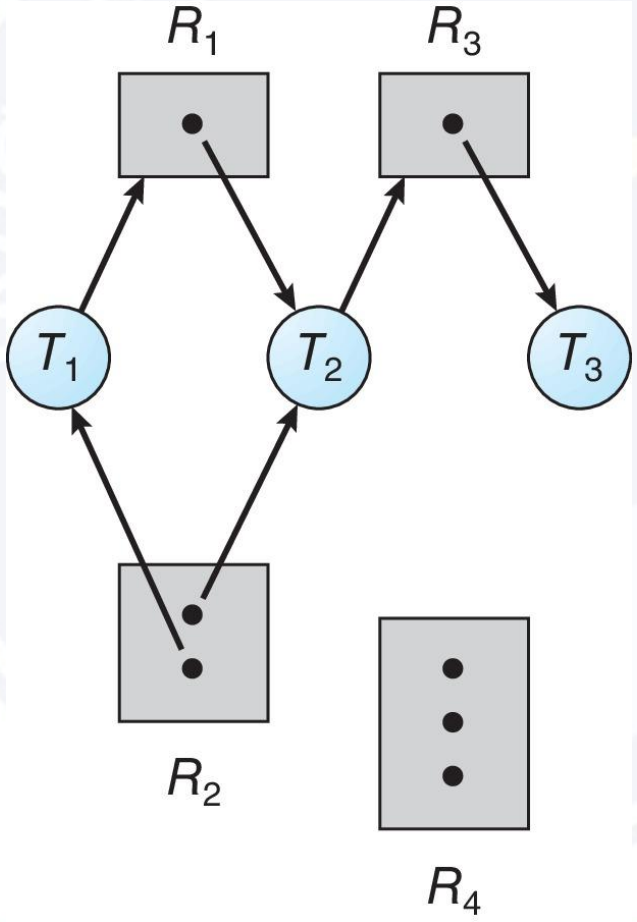
  - Pi đang giữ thực thể của Rj 

Giới thiệu về deadlock

- Đồ thị cấp phát tài nguyên (RAG: Resource-Allocation Graph)
  - RAG là đồ thị có hướng, với tập đỉnh V và tập cạnh E
  - Một tập hợp các đỉnh V và tập các cạnh E
  - Tập đỉnh V gồm 2 loại
    - $P = \{P_1, P_2, \ldots, P_n\}$, tập tất cả các tiến trình trong hệ thống
    - $R = \{R_1, R_2, \ldots, R_m\}$, tập tất cả các tài nguyên của hệ thống
  - Tập cạnh E gồm 2 loại :
    - - Cạnh yêu cầu (request edge): cạnh có hướng từ P_i -> R_j
    - - Cạnh cấp phát (assignment edge): cạnh có hướng từ R_j -> P_i

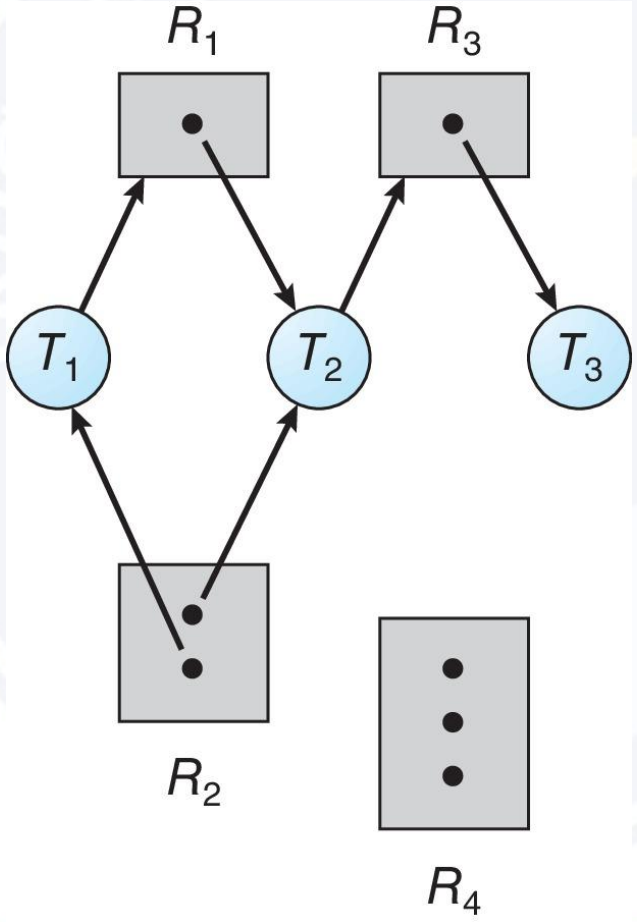
Giới thiệu về deadlock

- Ví dụ về RAG



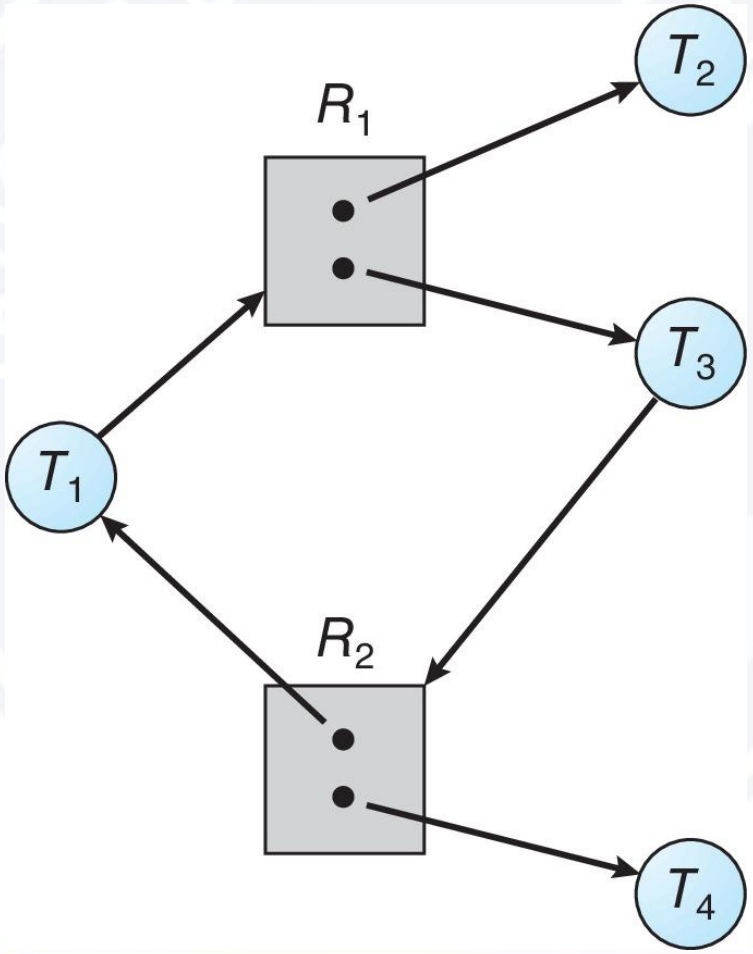
Giới thiệu về deadlock

- Ví dụ về RAG



Giới thiệu về deadlock

- Ví dụ về RAG



Giới thiệu về deadlock

- Suy luận cơ bản
  - Nếu đồ thị không chứa vòng (chu trình) -> không tắc nghẽn
  - Nếu đồ thị chứa vòng (chu trình) ->
    - Nếu tài nguyên chỉ có 1 thực thể, thì tắc nghẽn
    - Nếu tài nguyên có nhiều thực thể, có khả năng tắc nghẽn

Các phương thức quản lý deadlock

- Đảm bảo hệ thống sẽ không bao giờ rơi vào trạng thái tắc nghẽn:
  - Ngăn chặn tắc nghẽn
  - Tránh tắc nghẽn
- Cho phép hệ thống rơi vào trạng thái tắc nghẽn, sau đó phát hiện và phục hồi hệ thống

Các phương thức quản lý deadlock

- Bỏ qua việc xử lý tắc nghẽn, xem như hệ thống không bao giờ xảy ra tắc nghẽn. Được sử dụng bởi hầu hết các hệ điều hành, bao gồm cả UNIX

Phát hiện deadlock

- Cho phép hệ thống rơi vào trạng thái tắc nghẽn
- Giải thuật phát hiện tắc nghẽn
- Cách thức phục hồi

Ngăn chặn deadlock

- Ngăn chặn 4 điều kiện làm xuất hiện tắc nghẽn

1. Ngăn độc quyền truy xuất
   - đối với nonsharable resource (vd: printer): không làm được
   - đối với sharable resource (vd: các tập tin chỉ đọc): không cần thiết

Ngăn chặn deadlock

2. Ngăn giữ và đợi – phải đảm bảo rằng bất cứ khi nào tiến trình yêu cầu tài nguyên thì nó không được giữ bất kỳ tài nguyên nào khác
   - Đòi hỏi tiến trình yêu cầu và được cấp phát tất cả tài nguyên trước khi nó bắt đầu thực thi, hoặc cho phép tiến trình yêu cầu tài nguyên chỉ khi tiến trình không có tài nguyên được cấp phát
   - Nhược điểm :
     - Hiệu suất sử dụng tài nguyên thấp, tiến trình có thể rơi vào trạng thái đói tài nguyên

Ngăn chặn deadlock

3. Ngăn không trưng dụng (No preemption) – Nếu một tiến trình A đang giữ một số tài nguyên và đang yêu cầu tài nguyên khác mà không được cấp phát ngay lập tức thì:

- Cách 1: Hệ thống lấy lại mọi tài nguyên mà tiến trình đang giữ. Và tiến trình A chỉ bắt đầu lại khi có được các tài nguyên đã bị lấy lại cùng với tài nguyên đang yêu cầu.

Ngăn chặn deadlock

- Cách 2: Hệ thống sẽ xem xét tài nguyên mà tiến trình yêu cầu
  - Nếu tài nguyên được giữ bởi một tiến trình khác đang đợi thêm tài nguyên, thì tài nguyên này sẽ được lấy lại và cấp phát cho A.
  - Nếu tài nguyên giữ bởi tiến trình không đợi tài nguyên thì A phải đợi và tài nguyên của A bị lấy lại. Tuy nhiên hệ thống chỉ lấy lại các tài nguyên mà tiến trình khác yêu cầu.

Ngăn chặn deadlock

4. Ngăn đợi vòng quanh – đặt tất cả các loại tài nguyên vào thứ tự, và đòi hỏi mỗi tiến trình yêu cầu tài nguyên theo thứ tự tăng dần.

- Tập hợp các loại tài nguyên: R={R1, R2,…, Rn}
- Hàm ánh xạ F là hàm định nghĩa tứ tự trên tập các loại tài nguyên: F: R -> N
- Ví dụ:

  F(tape drive) = 1

  F(diskdrive) = 5

  F(printer) = 12

Ngăn chặn deadlock

4. Ngăn đợi vòng quanh – đặt tất cả các loại tài nguyên vào thứ tự, và đòi hỏi mỗi tiến trình yêu cầu tài nguyên theo thứ tự tăng dần.

- Tập hợp các loại tài nguyên: R={R1, R2,…, Rn}
- Hàm ánh xạ F là hàm định nghĩa tứ tự trên tập các loại tài nguyên: F: R -> N
- Ví dụ:

  F(tape drive) = 1

  F(diskdrive) = 5

  F(printer) = 12

Ngăn chặn deadlock

4. Ngăn đợi vòng quanh – đặt tất cả các loại tài nguyên vào thứ tự, và đòi hỏi mỗi tiến trình yêu cầu tài nguyên theo thứ tự tăng dần.
   - Mỗi tiến trình chỉ có thể yêu cầu thực thể của 1 loại tài nguyên theo thứ tự tăng dần (định nghĩa bởi hàm F) của loại tài nguyên.
   - Ví dụ:
     - Chuỗi yêu cầu thực thể hợp lệ: tape drive -> disk drive -> printer
     - Chuỗi yêu cầu thực thể không hợp lệ: disk drive -> tape drive
     - Khi một tiến trình yêu cầu 1 thực thể của loại tài nguyên R_j thì nó phải trả lại các tài nguyên R_i với $F(R_i) > F(R_j)$

Tránh deadlock

- Yêu cầu hệ thống
  - Mô hình đơn giản và hữu ích nhất là mỗi tiến trình sẽ khai báo số tài nguyên tối đa của mỗi loại mà tiến trình cần
  - Các thuật toán tránh tắc nghẽn tự động kiểm tra trạng thái cấp phát tài nguyên để đảm bảo rằng không bao giờ xảy ra điều kiện đợi vòng quanh
  - Trạng thái cấp phát tài nguyên được xác định bằng số tài nguyên còn lại, số tài nguyên đã được cấp phát, và yêu cầu tối đa của các tiến trình

Tránh deadlock

- Trạng thái an toàn và không an toàn
  - Khi một tiến trình yêu cầu một tài nguyên có sẵn, hệ thống phải quyết định cấp phát ngay lập tức nếu hệ thống đang ở trạng thái an toàn
  - Hệ thống là trạng thái an toàn nếu tồn tại một chuỗi tiến trình <P_1, P_2, …, P_n> trong hệ thống mà mỗi P_i, tài nguyên mà P_i yêu cầu vẫn có thể được thỏa mãn bởi tài nguyên sẵn có hiện hành cùng với tài nguyên được giữ bởi tất cả các P_j, với j<i
  - Khi Pi kết thúc, Pi+1 có được tài nguyên cần của nó, và cứ tiếp tục

Tránh deadlock

- Ví dụ trạng thái an toàn
  - Hệ thống có 12 ổ đĩa mềm và 3 tiến trình P_0, P_1, P_2. Tiến trình P_0 yêu cầu 10 ổ đĩa, P_1 yêu cầu 4 và P_2 yêu cầu 9.
  - Tại thời điểm T_0, P_0 được cấp 5, P_1 được cấp 2 và P_2 được cấp 2, khi đó hệ thống còn 3 ổ đĩa.

Tránh deadlock

- Ví dụ trạng thái an toàn
  - Ở thời điểm T_0 hệ thống đang trong trạng thái an toàn với chuỗi an toàn là $< P_1, P_0, P_2 >$
  - Ở thời điểm T_1, P_2 yêu cầu thêm một ổ đĩa và được cấp phát, khi này hệ thống sẽ rơi vào trạng thái không an toàn.
  - Lỗi là hệ thống đã cấp phát cho P_2 thay vì buộc P_2 phải đợi.

Tránh deadlock

- Suy luận cơ bản
  - Nếu hệ thống ở trạng thái an toàn -> không có tắc nghẽn
  - Nếu hệ thống ở trạng thái không an toàn -> có khả năng tắc nghẽn
  - Tránh tắc nghẽn đảm bảo rằng hệ thống không bao giờ rơi vào trạng thái không an toàn

Tránh deadlock

- Các giải thuật tránh tắc nghẽn
  - Tài nguyên có một thực thể
    - Sử dụng đồ thị cấp phát tài nguyên (RAG)
  - Tài nguyên có nhiều thực thể
    - Sử dụng giải thuật Banker

Tránh deadlock

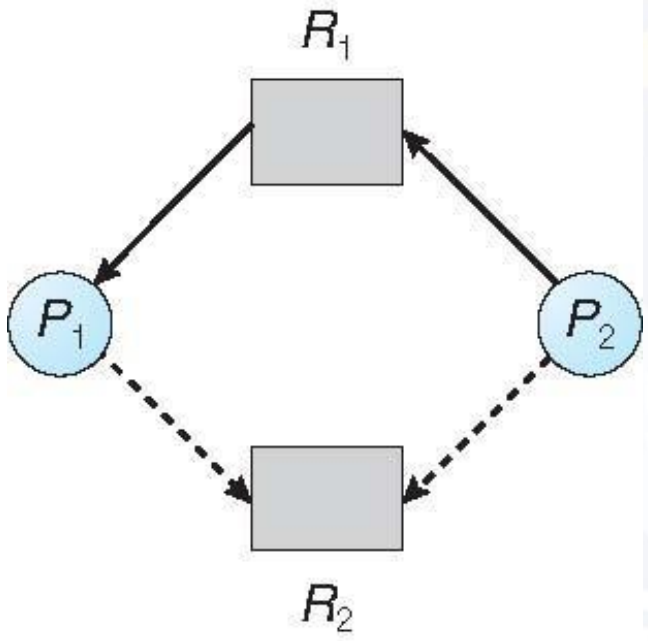
- Các giải thuật tránh tắc nghẽn
  - Sử dụng đồ thị cấp phát tài nguyên (RAG)
    - Cạnh đòi hỏi (claim edge) P_i -> R_j là cạnh cho biết tiến trình P_i có thể yêu cầu tài nguyên R_j , nó được biểu diễn bằng 1 đường nét đứt.
    - Cạnh đòi hỏi (claim edge) chuyển thành cạnh yêu cầu (request edge) khi một tiến trình yêu cầu tài nguyên.

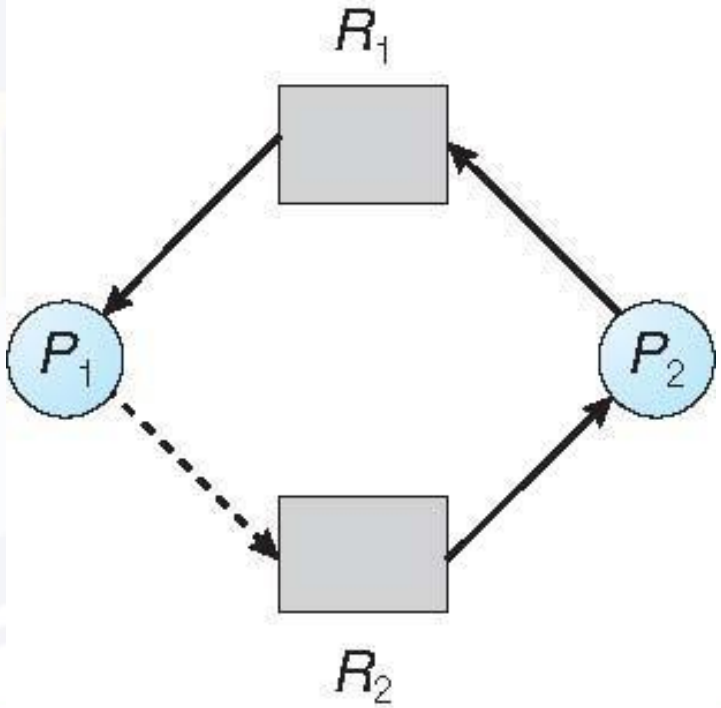
Tránh deadlock

- Các giải thuật tránh tắc nghẽn
  - Sử dụng đồ thị cấp phát tài nguyên (RAG)
    - Cạnh yêu cầu được chuyển thành cạnh cấp phát (assignment edge) khi tài nguyên được cấp cho tiến trình.
    - Khi tài nguyên được giải phóng bởi tiến trình, cạnh cấp phát chuyển lại thành cạnh đòi hỏi.
    - Các tài nguyên phải được gán 1 độ ưu tiên trong hệ thống.

Tránh deadlock

- Các giải thuật tránh tắc nghẽn
  - Sử dụng đồ thị cấp phát tài nguyên (RAG)





Tránh deadlock

- Các giải thuật tránh tắc nghẽn
  - Sử dụng đồ thị cấp phát tài nguyên (RAG)
    - Giả sử tiến trình P_i yêu cầu tài nguyên R_j
    - Yêu cầu được cấp phát chỉ khi việc chuyển từ cạnh yêu cầu thành cạnh cấp phát không tạo thành vòng trong đồ thị.

Tránh deadlock

- Giải thuật Banker
  - Áp dụng cho tài nguyên có nhiều thực thể
  - Mỗi tiến trình phải sử dụng độ ưu tiên cao nhất
  - Khi một tiến trình yêu cầu tài nguyên, nó có thể phải chờ
  - Khi một tiến trình có đầy đủ tài nguyên, nó phải trả lại chúng trong thời gian xác định

Tránh deadlock

- Giải thuật Banker
  - Gọi **n** là số tiến trình; **m** là số tài nguyên
  - Available: vector với độ dài **m**, là số thực thể đang khả dụng (có sẵn). Nếu available[j]=k, có k thực thể của tài nguyên R_j là khả dụng.
  - Max: ma trận nxm, số thực thể tối đa mà tiến trình yêu cầu. Nếu Max[i,j]=k, thì tiến trình P_i có thể yêu cầu nhiều nhất k thực thể của tài nguyên R_j

Tránh deadlock

- Giải thuật Banker
  - Allocation: ma trận nxm, số thực thể của tài nguyên đã cấp phát cho tiến trình. Nếu Allocation[i,j]=k, thì hiện hành tiến trình P_i được cấp phát k thực thể của tài nguyên R_j
  - Need: ma trận nxm, số thực thể tiến trình cần để hoàn thành tác vụ. Nếu Need[i,j]=k, thì tiến trình P_i cần thêm k thực thể của R_j để hoàn thành tác vụ

$$Need[i,j]=Max[i,j]-Allocation[i,j]$$

Tránh deadlock

- Giải thuật xác định trạng thái an toàn

1. Gọi **Work** và **Finish** là 2 vector với độ dài tương ứng là m và n. Khởi tạo:
   - **Work = Available**
   - **Finish [i] = false với i = 0, 1, …, n- 1**
2. Tìm i thỏa 2 điều kiện:
   - **Finish[i]=false**
   - **$Need_i$ <= Work**

Tránh deadlock

- Giải thuật xác định trạng thái an toàn
  - Nếu không tồn tại i như vậy thì qua Bước 4

3. **Work = Work + Allocation$_i$**

   Finish[i] = true

   Quay lại Bước 2

4. Nếu **Finish [i] == true** với tất cả **i**, thì hệ thống ở trạng thái an toàn

Tránh deadlock

- Giải thuật yêu cầu tài nguyên của tiến trình P_i
  - $Request_i$ – vector tài nguyên yêu cầu của tiến trình Pi. Nếu $Request_i[j]=k$ thì tiến trình Pi muốn k thực thể của tài nguyên Rj

1. Nếu $Request_i$ <= Needi tới Bước 2. Ngược lại, đưa ra lỗi vì tiến trình vượt quá đòi hỏi tối đa
2. Nếu $Request_i$ <= Available tới Bước 3. Ngược lại, Pi phải chờ vì tài nguyên không có sẵn

Tránh deadlock

- Giải thuật yêu cầu tài nguyên của tiến trình P_i

3. Giả sử hệ thống đã cấp phát cho Pi các tài nguyên mà nó yêu cầu và cập nhật tình trạng hệ thống như sau:
   - Available = Available – $Request_i$;
   - $Allocation_i$ = $Allocation_i$ + $Request_i$;
   - $Need_i$ = $Need_i$ – $Request_i$;

- Nếu trạng thái là an toàn -> tài nguyên được cấp phát cho P_i
- Nếu trạng thái là không an toàn -> P_i phải chờ, và trạng thái cấp phát tài nguyên cũ được phục hồi

Tránh deadlock

- Ví dụ về giải thuật Banker
  - 5 tiến trình: P_0 đến P_4
  - 3 loại tài nguyên:
    - A (10 thực thể); B (5 thực thể) và C (7 thực thể)
  - Hiện trạng tại thời điểm T_0:

	Allocation	***Max***	***Available***
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2		4 3 3

Tránh deadlock

- Ví dụ về giải thuật Banker
  - Ma trận ***Need*** được xác định bởi ***Max – Allocation***

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

  - Hệ thống ở trạng thái an toàn vì chuỗi **<P_1, P_3, P_4, P_0, P_2>** thỏa mãn điều kiện an toàn

Tránh deadlock

- Ví dụ về giải thuật Banker
  - P1 yêu cầu (1,0,2)
    - Kiểm tra $Request_1 \leq Available$ ((1,0,2) ≤ (3,3,2) ⇒ đúng)

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Tránh deadlock

- Ví dụ về giải thuật Banker
  - Thực hiện giải thuật xác định trạng thái an toàn, thấy rằng chuỗi <P1, P3, P4, P0, P2> đáp ứng yêu cầu an toàn
  - Yêu cầu (3,3,0) bởi P4 có thể được cấp phát không?
  - Yêu cầu (0,2,0) bởi P0 có thể được cấp phát không?

Giải thuật phát hiện tắc nghẽn

1. Gọi **Work** và **Finish** là các vector với độ dài tương ứng là m và n, được khởi tạo như sau:
   a) ***Work = available***
   b) Với mỗi i=1,2,…,n nếu ***Allocation*** $_i$ **≠ 0** thì ***Finish*** **[i] = *false***; ngược lại, ***Finish*** **[i] = *true***
2. Tìm i sao cho thỏa 2 điều kiện:
   a) ***Finish[i] == false***
   b) ***Request*** $_i$ ***≤ Work***

   Nếu i không tồn tại thì qua Bước 4

Giải thuật phát hiện tắc nghẽn

3. ***Work = Work + Allocation*** $_i$
   Finish[i] = true
   Quay lại Bước 2
4. Nếu ***Finish[i] == false*** với một vài i, 1 ≤ ***i*** ≤ ***n*** , thì hệ thống trong trạng thái tắc nghẽn. Ngoài ra, nếu ***Finish*** **[*i*] == *false*** thì P_i bị tắc nghẽn

Giải thuật phát hiện tắc nghẽn

- Ví dụ về giải thuật phát hiện tắc nghẽn
- Cho 5 tiến trình từ P0 đến P4; 3 tài nguyên: A (7 thực thể), B (2 thực thể) và C (6 thực thể)
- Hiện trạng tại thời điểm T0

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

THANK YOU!

665-667-669 Điện Biên Phủ, P. 1, Q. 3, Tp. HCM

1800 1568

www.vhu.edu.vn